



ESTRUCTURA DE COMPUTADORES

Tema 5. Arquitecturas avanzadas

Dpto. Arquitectura de Computadores y Automática
Universidad Complutense de Madrid



- ⊙ Introducción al procesamiento paralelo
- ⊙ Estructura de los multiprocesadores de memoria compartida
- ⊙ Estructura de los multiprocesadores de memoria distribuida
- ⊙ Arquitectura vectorial
- ⊙ Extensiones multimedia
- ⊙ GPUs

- ⊙ Bibliografía
 - ⊙ Cap 4 y Apéndice G de Hennessy & Patterson, 5th ed., 2012

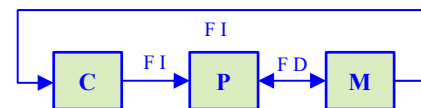


INTRODUCCIÓN AL PROCESAMIENTO PARALELO

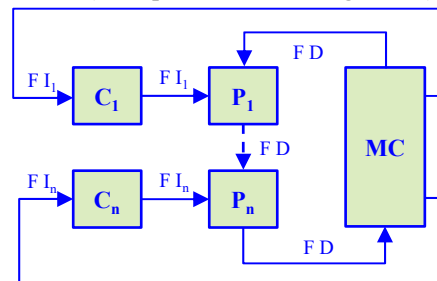
- © **Clasificación de Flynn de los computadores paralelos**
 - Flynn clasificó los computadores paralelos atendiendo al carácter Simple (S) o Múltiple (M) del Flujo de Instrucciones y el Flujo de Datos

CLASIFICACION DE FLYNN		Flujo Instrucciones	
		S	M
Flujo Datos	S	SISD	MISD
	M	SIMD	MIMD

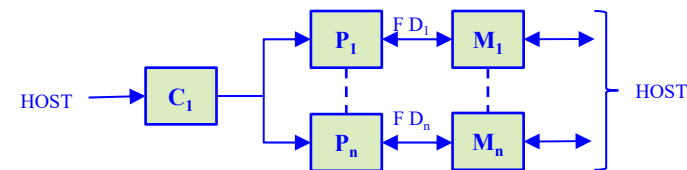
SISD (Single Instruction Single Data)



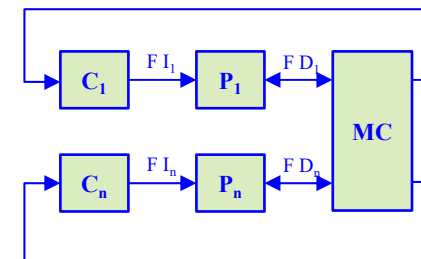
MISD (Multiple Instruction Single Data)



SIMD (Single Instruction Multiple Data)



MIMD (Multiple Instruction Multiple Data)



C = Control
P = Proceso (Unidad)
M = Memoria
MC = Memoria Compartida
FI = Flujo de Instrucciones
FD = Flujo de Datos

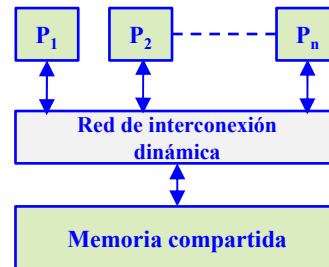


INTRODUCCIÓN AL PROCESAMIENTO PARALELO

⊙ Clasificación de los multiprocesadores por la ubicación de la memoria

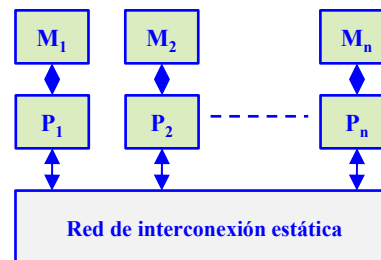
⊙ Multiprocesadores de memoria compartida

- Todos los procesadores acceden a una memoria común
- La comunicación entre procesadores se hace a través de la memoria



⊙ Multiprocesadores de memoria distribuida o multicomputadores

- Cada procesador tiene su propia memoria
- La comunicación se realiza por intercambio explícito de mensajes a través de una red

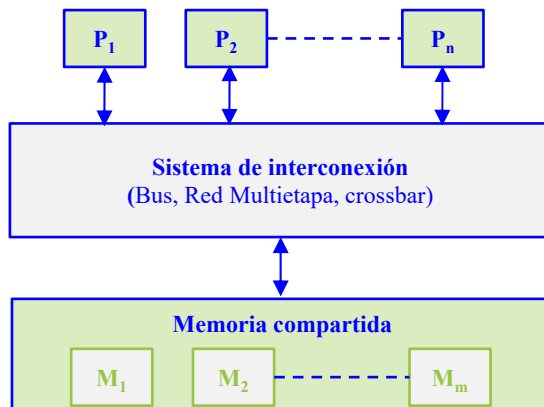




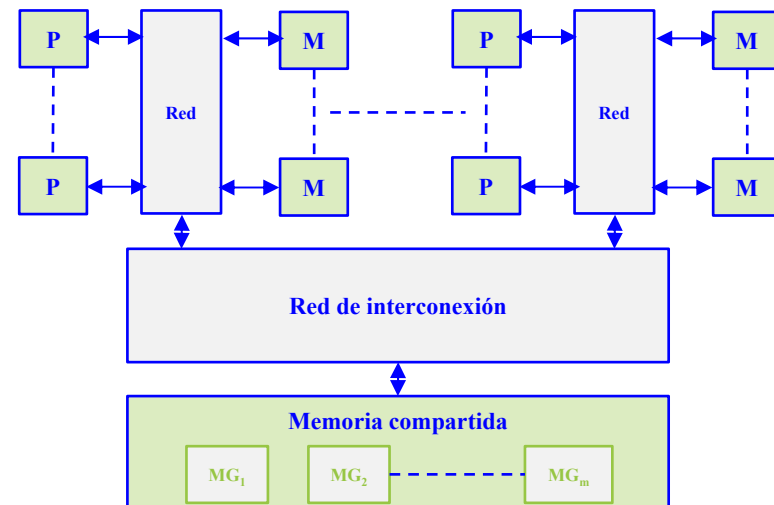
MULTIPROCESADORES DE MEMORIA COMPARTIDA

- ⊙ La mayoría de los multiprocesadores comerciales son del tipo UMA (*Uniform Memory Access*): todos los procesadores tienen igual tiempo de acceso a la memoria compartida.
- ⊙ En la arquitectura UMA los procesadores se conectan a la memoria a través de un bus, una red multietapa o un conmutador *crossbar* y disponen de su propia memoria caché.
- ⊙ Los procesadores tipo NUMA (*Non Uniform Memory Access*) presentan tiempos de acceso a la memoria compartida que dependen de la ubicación del elemento de proceso y la memoria.

Modelo UMA



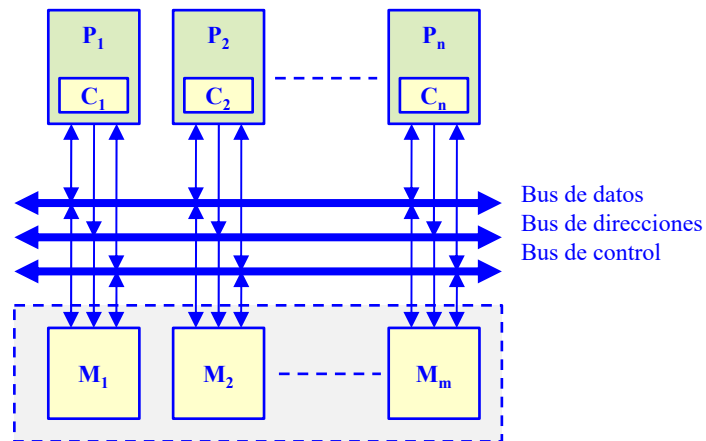
Modelo NUMA





INTERCONEXIÓN DE LOS PROCESADORES CON LA MEMORIA

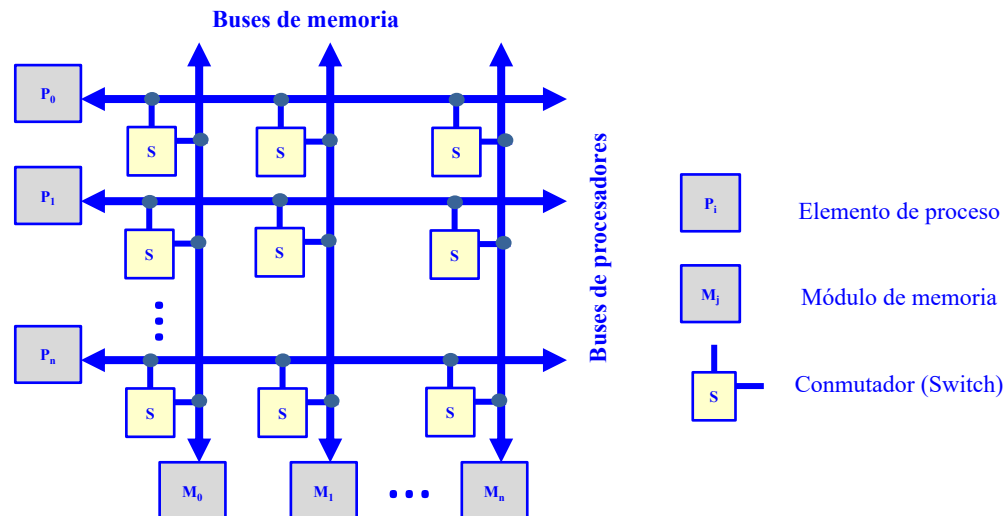
- ⊙ **Multiprocesadores de memoria compartida: conexión por bus compartido**
 - ⊙ Es la organización más común en los computadores personales y servidores
 - ⊙ El bus consta de líneas de dirección, datos y control para implementar:
 - El protocolo de transferencias de datos con la memoria
 - El arbitraje del acceso al bus cuando más de un procesador compite por utilizarlo.
 - ⊙ Los procesadores utilizan cachés locales para:
 - Reducir el tiempo medio de acceso a memoria, como en un monoprocesador
 - Disminuir la utilización del bus compartido.





INTERCONEXIÓN DE LOS PROCESADORES CON LA MEMORIA

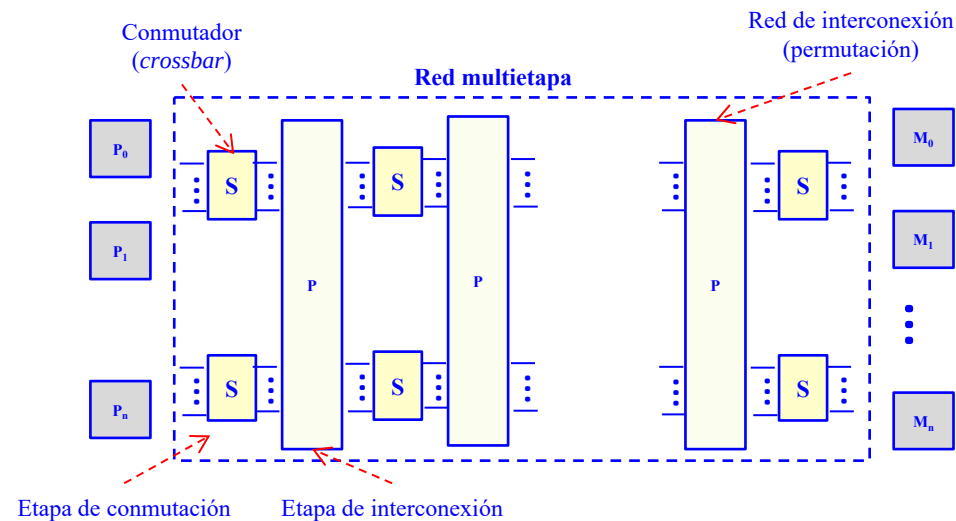
- ⊙ **Multiprocesadores de memoria compartida: conexión por conmutadores *crossbar***
 - ⊙ Cada procesador (P_i) y cada módulo de memoria (M_i) tienen su propio bus
 - ⊙ Existe un conmutador (S) en los puntos de intersección que permite conectar un bus de memoria con un bus de procesador
 - ⊙ Para evitar conflictos cuando más de un procesador pretende acceder al mismo módulo de memoria se establece un orden de prioridad
 - ⊙ Se trata de una red sin bloqueo con una conectividad completa pero de alta complejidad





INTERCONEXIÓN DE LOS PROCESADORES CON LA MEMORIA

- ⊙ **Multiprocesadores de memoria compartida: conexión por red multietapa**
 - ⊙ Representan una alternativa intermedia de conexión entre el bus y el *crossbar*.
 - ⊙ Es de menor complejidad que el *crossbar* pero mayor que el bus simple.
 - ⊙ La conectividad es mayor que la del bus simple pero menor que la del *crossbar*.
 - ⊙ Se compone de varias etapas alternativas de conmutadores simples y redes de interconexión.
 - ⊙ En general las redes multietapa responden al siguiente esquema:

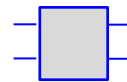




INTERCONEXIÓN DE LOS PROCESADORES CON LA MEMORIA

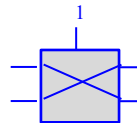
Ejemplo de red multietapa: red Omega (ω)

- Se trata de una red multietapa compuesta de conmutadores básicos de 2 entradas y 2 salidas

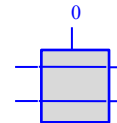


Conmutador básico

- Cada conmutador puede estar en dos estados: *paso directo* y *cruce*

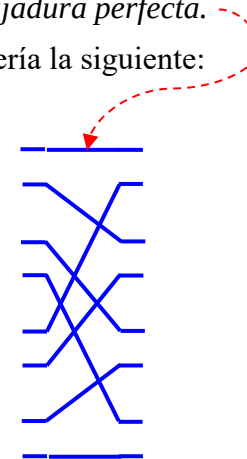
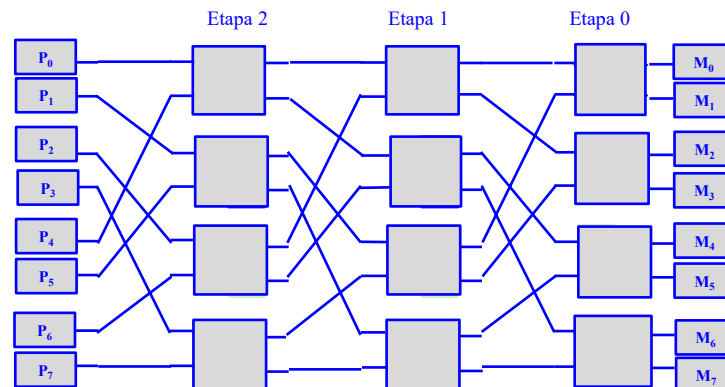


Cruce



Paso directo

- La interconexión entre etapas se realiza con un patrón fijo denominado *barajadura perfecta*.
- La red de 3 etapas que conecta 8 procesadores con 8 módulos de memoria sería la siguiente:

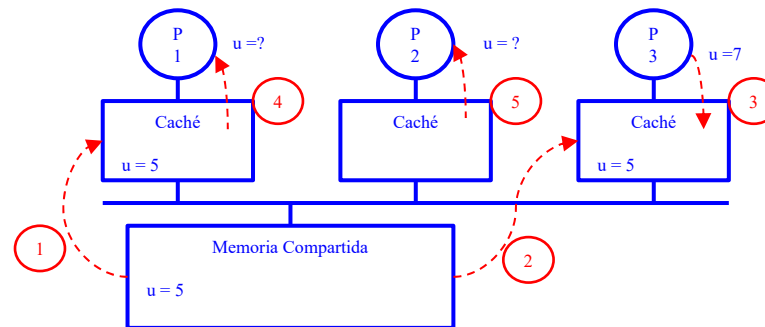




CONSISTENCIA DE MEMORIA CACHE

Problema de coherencia caché

- El problema de la coherencia caché en multiprocesadores surge por las operaciones de escritura.
- **Ejemplo:** secuencia de operaciones de acceso a memoria (1, 2, 3, 4, 5) realizadas por tres procesadores P1, P2 y P3 sobre la posición de memoria u :



- Caché con política *write-through*
 - La escritura realizada por el procesador $P3$ hará que el valor de u en la memoria principal sea 7. Sin embargo, el procesador $P1$ leerá el valor de u de su caché en vez de leer el valor correcto de la memoria principal.
- Caché con política *writeback*
 - En este caso el procesador $P3$ únicamente activará el bit de modificado en el bloque de la caché en donde tiene almacenado u y no actualizará la memoria principal.
 - Sólo cuando ese bloque de la caché sea reemplazado se actualizará la memoria principal.
 - No será sólo $P1$ el que leerá el valor antiguo, sino que cuando $P2$ intente leer u y se produzca un fallo en la caché, también leerá el valor antiguo de la memoria principal



CONSISTENCIA DE MEMORIA CACHE

⦿ Solución de la coherencia caché

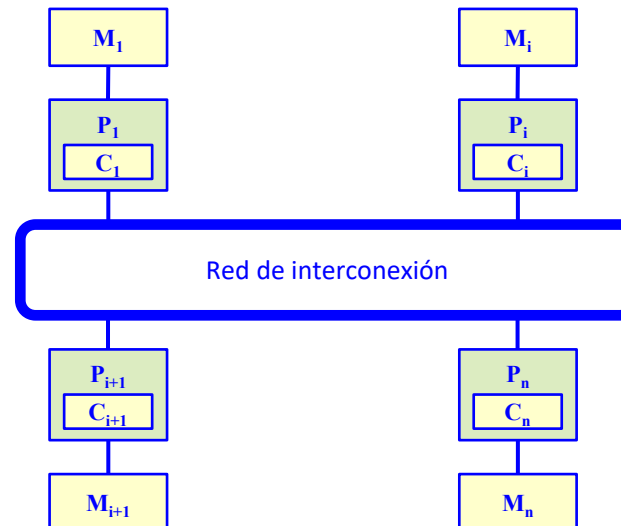
- ⦿ Existen dos formas de abordar el problema de la coherencia caché.
 - Software, lo que implica la realización de compiladores que eviten la incoherencia entre cachés de datos compartidos.
 - Hardware que mantengan de forma continua la coherencia en el sistema, siendo además transparente al programador.
- ⦿ Podemos distinguir también dos tipos de sistemas multiprocesadores
 - Sistemas basados en un único bus: se utilizan protocolos de sondeo o *snoopy* que analizan el bus para detectar incoherencia. Cada nodo procesador tendrá los bits necesarios para indicar el estado de cada línea de su caché y así realizar las transacciones de coherencia necesarias según lo que ocurra en el bus en cada momento.
 - Sistemas con redes multietapa: protocolo basado en directorio, que consiste en la existencia de un directorio común donde se guardan el estado de validez de las líneas de las cachés, de manera que cualquier nodo puede acceder a este directorio común.



MULTIPROCESADORES DE MEMORIA DISTRIBUIDA

Multicomputadores

- Los multiprocesadores de memoria compartida presentan algunas desventajas:
 - Se necesitan técnicas de sincronización para acceder a las variables compartidas
 - La contención en la memoria puede reducir significativamente la velocidad.
 - No son fácilmente escalables a un gran número de procesadores.
- Un **multicomputador** consta de un conjunto de procesadores conectados por una red
- Cada procesador tiene su propia memoria local, incluida la caché, y se comunican por paso de mensajes a través de la red





MULTIPROCESADORES DE MEMORIA DISTRIBUIDA

⊙ Propiedades de los multicomputadores

- ⊙ El número de nodos puede ir desde algunas decenas hasta varios miles (o más).
- ⊙ La arquitectura de paso de mensajes tiene ventajas sobre la de memoria compartida cuando el número de procesadores es grande.
- ⊙ El número de canales físicos entre nodos suele oscilar entre cuatro y ocho.
- ⊙ Esta arquitectura es directamente escalable y presenta un bajo coste para sistemas grandes.
- ⊙ Un problema se especifica como un conjunto de procesos que se comunican entre sí y que se hacen corresponder sobre la estructura física de procesadores.
- ⊙ El tamaño de un proceso viene determinado por su *granularidad*:

$$\text{Granularidad} = \frac{\text{Tiempo de cálculo}}{\text{Tiempo de comunicación}}$$

- ⊙ Al reducirse la granularidad, la sobrecarga de comunicación de los procesos aumenta.
- ⊙ Por ello, la granularidad empleada en este tipo de máquinas suele ser media o gruesa.
- ⊙ El programa a ejecutar debe de ser intensivo en cálculo, no intensivo en operaciones de entrada/salida o de paso de mensajes



TOPOLOGÍA DE LAS REDES ESTÁTICAS DE INTERCONEXIÓN

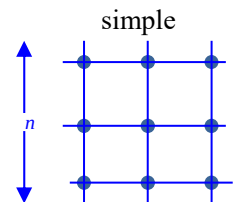
- ⊙ **Redes estáticas de interconexión en multicomputadores**
 - ⊙ Los multicomputadores utilizan redes estáticas con enlaces directos entre nodos
 - ⊙ Cuando un nodo recibe un mensaje lo procesa si viene dirigido a dicho nodo
 - ⊙ Si el mensaje no va dirigido al nodo receptor lo reenvía a otro por alguno de sus enlaces de salida siguiendo un protocolo de encaminamiento.
 - ⊙ Las propiedades más significativas de una red estática son las siguientes:
 - **Topología de la red:** determina el patrón de interconexión entre nodos.
 - **Diámetro de la red:** distancia máxima de los caminos más cortos entre dos nodos de la red.
 - **Latencia:** retardo de tiempo en el peor caso para un mensaje transferido a través de la red.
 - **Ancho de banda:** Transferencia máxima de datos en Mbytes/segundo.
 - **Escalabilidad:** posibilidad de expansión modular de la red.
 - **Grado de un nodo:** número de enlaces o canales que inciden en el nodo.
 - **Algoritmo de encaminamiento:** determina el camino que debe seguir un mensaje desde el nodo emisor al nodo receptor.



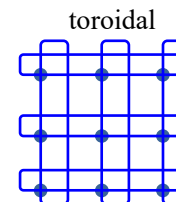
TOPOLOGÍA DE LAS REDES ESTÁTICAS DE INTERCONEXIÓN

Algunas topología de las redes estáticas

• Mallas

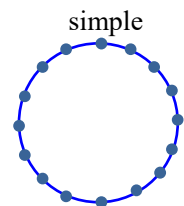


$$\begin{aligned} \text{diámetro} &= 2 \cdot (n-1) \\ \text{grado} &= 4 \\ n^{\circ} \text{ nodos} &= n^2 \\ n^{\circ} \text{ enlaces} &= 2 \cdot n \cdot (n-1) \end{aligned}$$

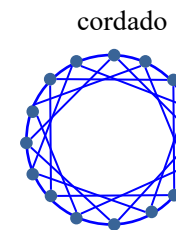


$$\begin{aligned} \text{diámetro} &= 2 \cdot \lfloor n/2 \rfloor \\ \text{grado} &= 4 \\ n^{\circ} \text{ nodos} &= n^2 \\ n^{\circ} \text{ enlaces} &= 2n^2 \end{aligned}$$

• Anillos

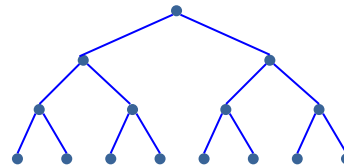


$$\begin{aligned} \text{diámetro} &= n-1 \\ \text{grado} &= 2 \\ n^{\circ} \text{ nodos} &= n \\ n^{\circ} \text{ enlaces} &= n-1 \end{aligned}$$



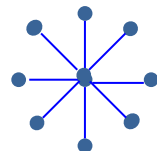
$$\begin{aligned} \text{diámetro} &= 3 \\ \text{grado} &= 4 \\ n^{\circ} \text{ nodos} &= n \\ n^{\circ} \text{ enlaces} &= 3n \end{aligned}$$

• Árbol



$$\begin{aligned} \text{diámetro} &= 2(k-1) \\ \text{grado} &= 4 \\ n^{\circ} \text{ nodos} &= 2^k - 1 \\ n^{\circ} \text{ enlaces} &= 2^k - 2 \\ k &= n^{\circ} \text{ niveles} \end{aligned}$$

• Estrella



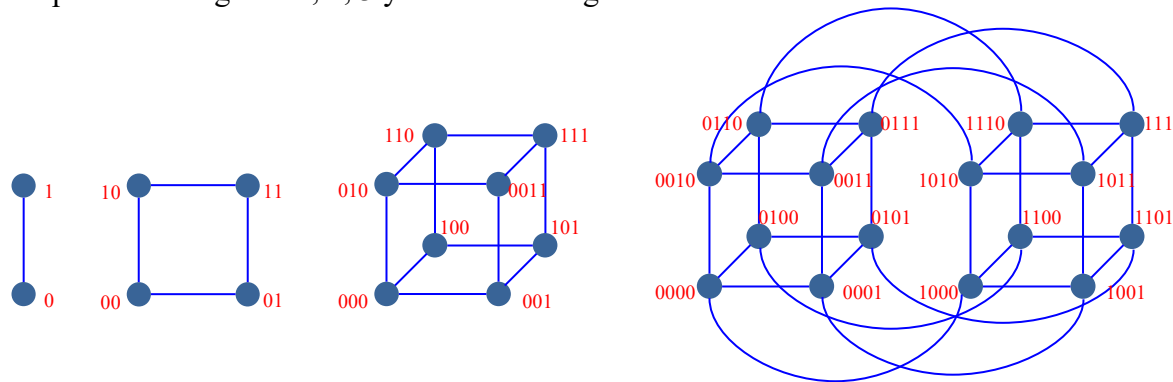
$$\begin{aligned} \text{diámetro} &= 2 \\ \text{grado} &= n-1 \\ n^{\circ} \text{ nodos} &= n \\ n^{\circ} \text{ enlaces} &= n-1 \end{aligned}$$



TOPOLOGÍA DE LAS REDES ESTÁTICAS DE INTERCONEXIÓN

⊙ Hiper cubos

- ⊙ Un *cubo- n* o hipercubo de dimensión n consta de $N=2^n$ nodos extendidos a lo largo de n dimensiones.
- ⊙ El grado vale n
- ⊙ El diámetro también vale n
- ⊙ Existen n caminos disjuntos entre cualquier par de nodos
- ⊙ Cada nodo se etiqueta con un número binario de n bits de tal modo asignados que dos vértices conectados se diferencian en un solo bit.
- ⊙ Los hipercubos de grado 1, 2, 3 y 4 serían los siguientes:





⊙ SIMD (Single Instruction Multiple Data).

- ⊙ Una operación (codificada como una sola instrucción de Lenguaje Máquina) se ejecuta sobre un conjunto de datos (en contraposición a SISD).

- ⊙ Ejemplo:

- ⊙ En lenguaje matemático: $\vec{V3} = \vec{V2} + \vec{V1}$
- ⊙ En LAN:

```
for (i = 0; i < N; i++)  
    V3(i) = V2(i) + V1(i);
```
- ⊙ En LM: `ADDV V3,V2,V1`

⊙ Arquitectura SIMD: puede explotar una cantidad importante de paralelismo de datos en:

- ⊙ Aplicaciones de ciencia/ingeniería con abundante cálculo matricial (ámbito tradicional)
- ⊙ Nuevas aplicaciones: gráficos, visión artificial, comprensión de voz, ...



- ⊙ Eficiencia energética de SIMD: puede ser ventajosa frente a MIMD
 - ⊙ Sólo es preciso hacer un “fetch” para operar sobre varios datos.
 - ⊙ Ahorro de energía atractivo en dispositivos portátiles

- ⊙ En SIMD el programador sigue “pensando” básicamente en un flujo secuencial de instrucciones.

- ⊙ Soporte arquitectónico para explotar paralelismo SIMD
 - ⊙ Arquitectura vectorial
 - ⊙ Extensiones SIMD (extensiones multimedia)
 - ⊙ Graphics Processor Units (GPUs)



ARQUITECTURA VECTORIAL

⊙ Ideas básicas

- ⊙ Leer conjuntos de datos sobre “registros vectoriales”
- ⊙ Operar sobre el contenido de estos registros
- ⊙ Almacenar los resultados finales en memoria
 - Usar los registros vectoriales para ocultar la latencia de memoria

⊙ Características de las operaciones vectoriales

- ⊙ Secuencias de cálculos independientes → Ausencia de riesgos
- ⊙ Alto contenido semántico
 - Una instrucción → Muchas operaciones
- ⊙ Patrón de accesos a memoria conocido
- ⊙ Explotación eficiente de memoria entrelazada
- ⊙ Disminución de instrucciones de salto (un bucle completo puede transformarse en una instrucción)
 - Reducción conflictos de control



ARQUITECTURA VECTORIAL

◎ En el principio... Seymour Cray – CDC 6600 (1963)



- ❑ No es una arquitectura vectorial, pero...
 - o Muy segmentada, con palabra de 60 bits
 - o MP de 128 Kword con 32 bancos
 - o 10 FUs (paralelas, no segmentadas)
 - PF: sumador, 2 multiplicadores, divisor
 - o Control cableado
 - o Planificación dinámica de instrucciones (scoreboard)
 - o 10 procesadores de E/S
 - o Clock 10 MHz
 - Muy rápido para la época
 - Suma PF en 4 ciclos
 - o >400,000 transistores, 750 sq. ft. (~70 m²), 5 tons, 150 KW, refrigeración freon
 - o Máquina más rápida del mundo durante 5 años (hasta el 7600)
 - o Ventidas >100 (a \$7-10M c.u.)



ARQUITECTURA VECTORIAL

El Cray-1 (1976)

- ⊙ Unidad escalar
 - ⊙ Arquitectura Load/Store
- ⊙ Extensión Vectorial
 - ⊙ Registros Vectoriales
 - ⊙ Instrucciones Vectoriales
- ⊙ Implementación
 - ⊙ Control cableado
 - ⊙ UF muy segmentadas
 - ⊙ Memoria entrelazada
 - ⊙ Sin cache de datos
 - ⊙ Sin memoria virtual





ARQUITECTURA VECTORIAL

⊙ Estructura de un procesador vectorial: VMIPS

Funcionalmente
equivalente a un pipe: un
dato por ciclo, después de
latencia inicial (12 ciclos).

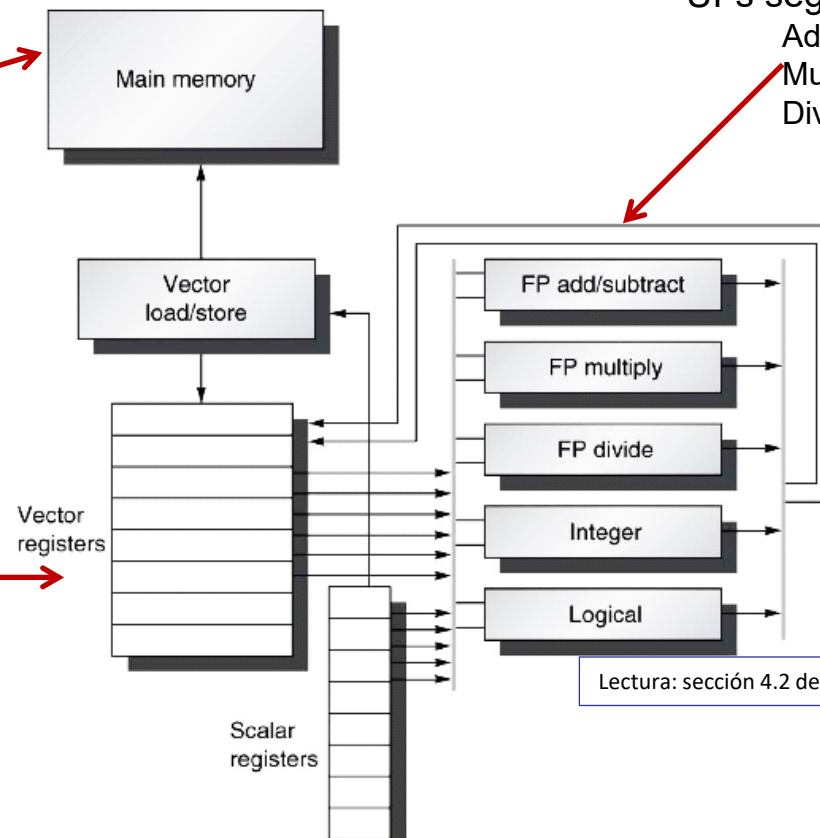
UFs segmentadas:

Add 6 etapas

Mul 7 etapas

Div 20 etapas

8 registros vectoriales
1 reg = 64 elementos
1 elemento = 64 bits
16 read, 8 write ports



Lectura: sección 4.2 de H&P 5th ed.



ARQUITECTURA VECTORIAL: REPERTORIO VMIPS (1)

- ⊙ Operaciones vectoriales aritméticas sobre registros
- ⊙ Instrucciones especiales de carga/almacenamiento de vectores (LV,SV)
- ⊙ Modos de direccionamiento especiales para vectores no contiguos. Ejemplos
 - ⊙ LVWS: Load Vector With Stride. Carga elementos equiespaciados a una cierta distancia
 - ⊙ LVI: Load Vector using Index. El contenido de un registro vectorial indica las posiciones de los elementos a cargar.
- ⊙ Registros especiales de longitud vectorial (VLR) y máscara (VM)
 - ⊙ Reg VLR: Indica la longitud de los vectores a procesar (≤ 64)
 - ⊙ Reg VM: Registro de 64 bits. Para las posiciones con VM a cero, la operación no se ejecuta.
 - Ejecución selectiva de operaciones sobre componentes



ARQUITECTURA VECTORIAL: REPERTORIO VMIPS (2)

Instruction	Operands	Function
ADDVV.D	V1,V2,V3	Add elements of V2 and V3, then put each result in V1.
ADDVS.D	V1,V2,F0	Add F0 to each element of V2, then put each result in V1.
SUBVV.D	V1,V2,V3	Subtract elements of V3 from V2, then put each result in V1.
SUBVS.D	V1,V2,F0	Subtract F0 from elements of V2, then put each result in V1.
SUBSV.D	V1,F0,V2	Subtract elements of V2 from F0, then put each result in V1.
MULVV.D	V1,V2,V3	Multiply elements V2 and V3, then put each result in V1.
MULVS.D	V1,V2,F0	Multiply each element of V2 by F0, then put each result in V1.
DIVVV.D	V1,V2,V3	Divide elements of V2 by V3, then put each result in V1.
DIVVS.D	V1,V2,F0	Divide elements of V2 by F0, then put each result in V1.
DIVSV.D	V1,F0,V2	Divide F0 by elements of V2, then put each result in V1.
LV	V1,R1	Load vector register V1 from memory starting at address R1.
SV	R1, V1	Store vector register V1 into memory starting at address R1.
LVWS	V1,(R1,R2)	Load V1 from address at R1 with stride in R2 (i.e .. $R1 + i \times R2$).
SVWS	(R1,R2),V1	Store V1 to address at R1 with stride in R2 (i.e .. $R1 + i \times R2$).
LVI	V1,(R1+V2)	Load V1 with vector whose elements are at $R1 + V2(i)$ (i.e., V2 is an index).
SVI	(R1+V2) ,V1	Store V1 to vector whose elements are at $R1 + V2(i)$ (i.e., V2 is an index).



ARQUITECTURA VECTORIAL: REPERTORIO VMIPS (3)

Instruction	Operands	Function
CVI	V1,R1	Create an index vector by storing the values 0, 1 x R1, 2 x R1, ... ,63 x R1 into V1.
S--VV.D S--VS.D	V1, V2 V1, F0	Compare the elements (EQ, NE, GT, LT, GE, LE) in V1 and V2. If condition is true, put a 1 in the corresponding bit vector: otherwise put 0. Put resulting bit vector in vector mask register (VM). The instruction S--VS.D performs the same compare but using a scalar value as one operand
POP	R1,VM	Count the 1s in vector-mask register VM and store count in R1.
CVM		Set the vector-mask register to all 1s.
MTC1 MFC1	VLR,R1 R1,VLR	Move contents of R1 to vector-length register VL. Move the contents of vector-length register VL to R1.
MVTM MVFM	VM,F0 F0,VM	Move contents of F0 to vector-mask register VM. Move contents of vector-mask register VM to F0.



CÓDIGO ESCALAR VS. CÓDIGO VECTORIAL

◎ $Y = a * X + Y$ (AXPY, Sup: vectores de 64 elementos)

◎ Versión escalar

Loop:

L.D	F0, a	; load scalar a
DADDIU	R4, Rx, #512	; last address to load
L.D	F2, 0(Rx)	; Load X[i]
MUL.D	F2, F2, F0	; A x X[i]
L.D	F4, 0(Ry)	; Load Y[i]
ADD.D	F4, F4, F2	; A x X[i] + Y[i]
S.D	F4, 0(Ry)	; Store Y[i]
DADDIU	Rx, Rx, #8	; increment index X
DADDIU	Ry, Ry, #8	; increment index Y
DSUBU	R20, R4, Rx	; loop exhausted?
BNEZ	R20, Loop	

◎ Versión vectorial

L.D	F0, a	; load scalar a
LV	V1, Rx	; Load vector X
MULVS.D	V2, V1, F0	; Vector-scalar multiply
LV	V3, Ry	; Load vector Y
ADDVV.D	V4, V2, V3	; Vector addition
SV	V4, Ry	; Store result Y

◎ Instrucciones ejecutadas: ~ 580 vs. 6 !!





⊙ Dependiente básicamente de tres factores

- ⊙ Longitud de los vectores operandos
- ⊙ Riesgos estructurales: las UF necesarias están ocupadas, no hay puertos del BR disponibles
- ⊙ Dependencias de datos

⊙ Velocidad de procesamiento

- ⊙ Las FUs del VMIPS consumen (y producen) un elemento por ciclo de reloj
- ⊙ El tiempo de ejecución de una operación vectorial es aproximadamente igual a la longitud del vector

⊙ Convoy

- ⊙ Se denomina así a un conjunto de (una o varias) instrucciones vectoriales que potencialmente pueden ejecutarse juntas (ausencia de riesgos estructurales). Pueden tener riesgos LDE.



TIEMPO DE EJECUCIÓN

⦿ Convojes: ejemplo

1: LV	V1, Rx	; load vector X
2: MULVS.D	V2, V1, F0	; vector-scalar multiply
3: LV	V3, Ry	; load vector Y
4: ADDVV.D	V4, V2, V3	; Vector addition
5: SV	V4, Ry	; Store result Y

⦿ Conflictos:

- 1 y 2 no tienen conflictos estructurales
- 3 tiene conflicto estructural con 1 (una sola unidad de Load/Store)
- 4 no tiene conflictos estructurales con 3
- 5 tiene conflicto estructural con 3

⦿ Convojes resultantes

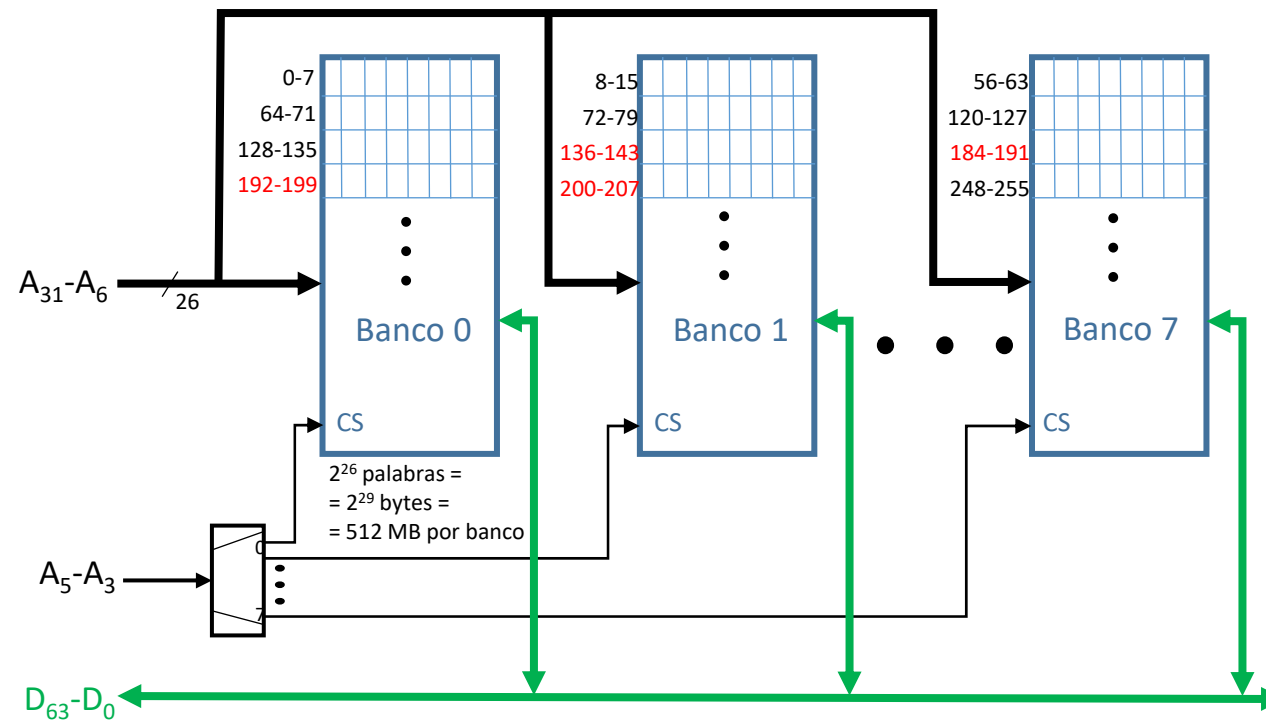
- 1. Formado por LV y MULVS.D
- 2. Formado por LV y ADDVV.D
- 3. Formado por SV



MEMORIA ENTRELAZADA

Memoria con entrelazamiento de orden bajo (a nivel de palabra) con 8 bancos

- Longitud de palabra: 8 bytes
- Palabras consecutivas están en bancos consecutivos.
- Cada 8 palabras se vuelve a usar el mismo banco.





ENCADENAMIENTO

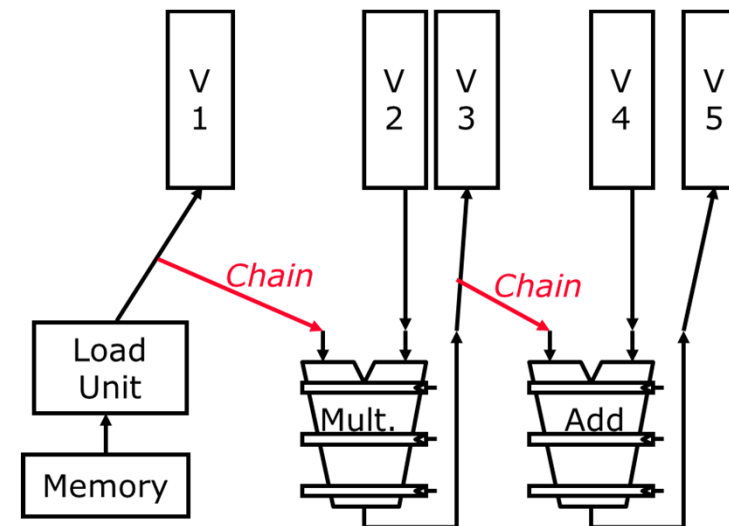
- Operaciones dependientes:
Tratamiento de dependencias RAW

- Problema

LV	V1
MULVV.D	V3, V1, V2
ADDVV.D	V5, V3, V4

- Solución:

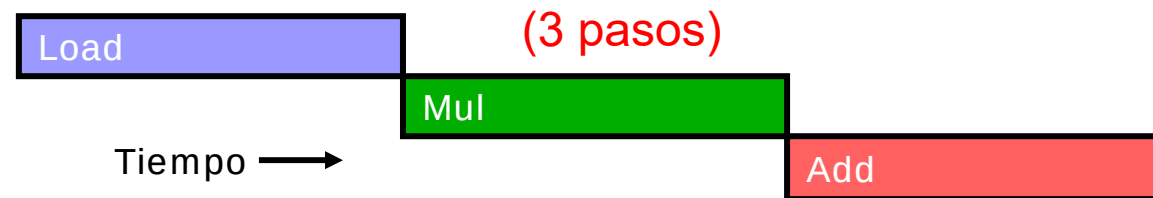
- Cray-1. Extensión del concepto de anticipación de operandos $\Rightarrow$ Encadenamiento de operaciones (chaining)
- 3 operaciones, pero 1 paso
- En proc modernos: “encadenamiento flexible”



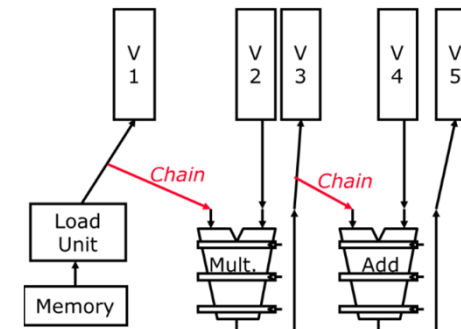
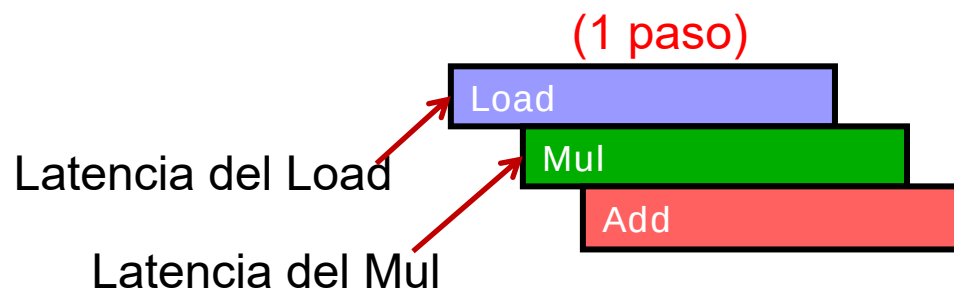


ENCADENAMIENTO

- ⊙ Sin encadenamiento: esperar hasta que se haya calculado el último elemento de la operación anterior



- ⊙ Con encadenamiento: Una instrucción puede comenzar cuando está disponible el primer elemento de la operación de la que depende





OPERACIONES CONDICIONALES: REGISTRO DE MÁSCARA

⊙ Ejecutar

```
for (i = 0; i < 64; i=i+1)
    if (X[i] != 0)
        X[i] = X[i] - Y[i];
```

⊙ El registro de máscara vectorial permite omitir las operaciones sobre los elementos que no cumplen la condición

LV	V1,Rx	;load vector X into V1
LV	V2,Ry	;load vector Y
L.D	F0,#0	;load FP zero into F0
SNEVS.D	V1,F0	;sets VM(i) to 1 if V1(i) != F0
SUBVV.D	V1,V1,V2	;subtract <u>under vector mask</u>
SV	Rx,V1	;store the result in X

⊙ Obviamente, el rendimiento se reduce

- ⊙ Se consume el mismo tiempo
- ⊙ Se ejecutan menos operaciones útiles



VECTORES NO CONSECUTIVOS EN MEMORIA

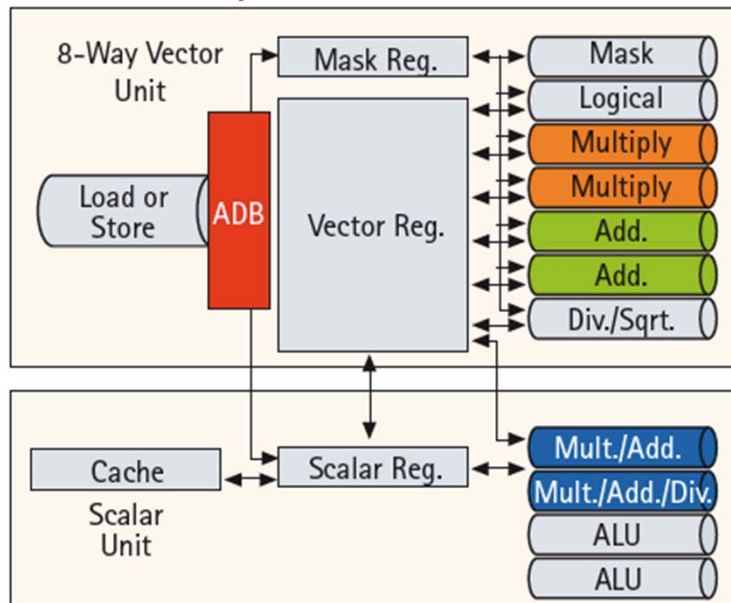
- ⊙ Ejemplo: Producto de matrices

```
for (i = 0; i < 100; i=i+1)
  for (j = 0; j < 100; j=j+1) {
    A[i][j] = 0.0;
    for (k = 0; k < 100; k=k+1)
      A[i][j] = A[i][j] + B[i][k] * D[k][j];
  }
```
- ⊙ Las matrices están ordenadas por filas
 - ⊙ Para vectorizar el producto de filas de B por columnas de D
 - Acceso a elementos consecutivos de B
 - Acceso a elementos de D separados por 100 palabras de distancia
- ⊙ Soporte: Instrucciones de load/store con “stride” (espaciamiento)
 - ⊙ Problema: Repetición de acceso al mismo banco de memoria antes del fin de la op anterior
 - ⊙ Acceso libre de conflicto si:
 - $mcm(\text{espaciamiento}, n^{\circ} \text{ bancos}) / \text{espaciamiento} \geq T$ acceso a banco



EJ. DE SUPERCOMPUTADOR VECTORIAL: NEC SX-9

Esquema de 1 CPU



Introducido en 2008

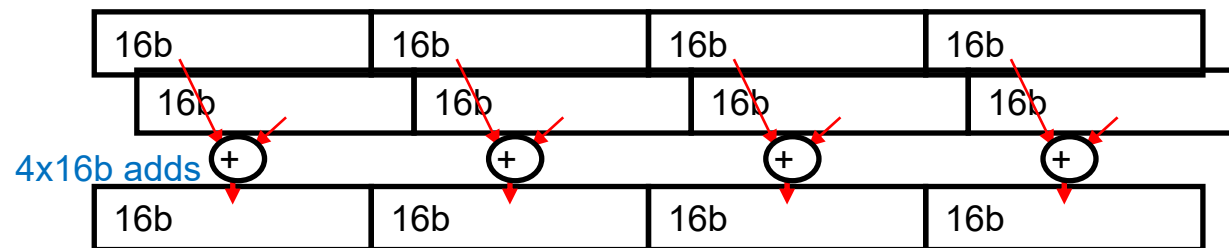
- ⊙ Tecnología 65nm CMOS
- ⊙ Unidad Vectorial (3.2 GHz)
 - ⊙ 8 foreground VRegs + 64 background VRegs (256x64-bit elementos/VReg)
 - ⊙ UFs de 64-bits: 2 multiply, 2 add, 1 divide/sqrt, 1 logical, 1 mask unit
 - ⊙ 8 vías (32+ FLOPS/cycle, 100+ GFLOPS pico por CPU)
 - ⊙ 1 load or store unit (8 x 8-byte accesos/ciclo)
- ⊙ Unidad escalar (1.6 GHz)
 - ⊙ Superescalar 4 vías O-o-O
 - ⊙ 64KB I-cache, 64KB data cache

- ❑ AB Memoria: 256GB/s por CPU
 - o Hasta 16 CPUs y hasta 1TB de DRAM forman un **nodo** con mem compartida
 - o AB total: 4TB/s a la DRAM compartida
- ❑ Hasta 512 nodos conectados mediante enlaces de 128 GB/s (Paso de mensajes entre nodos)



EXTENSIONES SIMD

- ⊙ También conocidas como “extensiones multimedia”
- ⊙ Observación: las aplicaciones multimedia suelen operar sobre datos de menor anchura que las UFs y los registros disponibles
- ⊙ Idea: Realizar varias operaciones a la vez
 - ⊙ Por ejemplo: desconectar la cadena de propagación de carries
 - ⊙ Una sola instrucción de suma opera sobre varios elementos almacenados en un registro → equivale a una op vectorial, aunque con un vector de pocos elementos



- ⊙ Limitaciones, comparado con op vectoriales:
 - ⊙ La longitud de los vectores se codifica en el Cod_op
 - ⊙ No existe direccionamiento sofisticado (stride, gather,...)
 - ⊙ No hay registro de máscara



EXTENSIONES SIMD

- ⊙ Implementaciones:
 - ⊙ Intel MMX (1996)
 - Ocho ops enteras de 8-bit o cuatro ops enteras de 16-bit
 - ⊙ Streaming SIMD Extensions (SSE) (1999-2007)
 - Ocho ops enteras de 16-bit
 - Cuatro ops enteras/FP de 32-bit o dos ops enteras/FP de 64-bit
 - ⊙ Advanced Vector Extensions (AVX)(2010)
 - Cuatro ops enteras/FP de 64-bit
 - ⊙ Los operandos deben ser consecutivos y en posiciones de memoria alineadas



EXTENSIONES SIMD

🎯 Ejemplo: Instrucciones AVX para la arquitectura x86

4 operandos de 64 bits

AVX Instruction	Description
VADDPD	Add four packed double-precision operands
VSUBPD	Subtract four packed double-precision operands
VMULPD	Multiply four packed double-precision operands
VDIVPD	Divide four packed double-precision operands
VFMADDPD	Multiply and add four packed double-precision operands
VFMSUBPD	Multiply and subtract four packed double-precision operands
VCMPxx	Compare four packed double-precision operands for EQ, NEQ, LT, LE, GT, GE, ..
VMOVAPD	Move aligned four packed double-precision operands
VBROADCASTSD	Broadcast one double-precision operand to four locations in a 256-bit register

Como la longitud de los operandos va indicada en el Cod_op, puede dar la impresión de que el nº de instr de las “extensiones multimedia” es mayor de lo que en realidad es.



EJ. CÓDIGO CON EXTENSIONES SIMD EN MIPS

- Sup: Añadimos instrucciones multimedia SIMD de 256 bits al MIPS (".4D" → operaciones sobre 4 operandos de 64 bits a la vez)
- Código para DAXPY:

	L.D F0,a	;load scalar a
	MOV F1, F0	;copy a into F1 for SIMD MUL
	MOV F2, F0	;copy a into F2 for SIMD MUL
	MOV F3, F0	;copy a into F3 for SIMD MUL
	DADDIU R4,Rx,#512	;last address to load
Loop:	L.4D F4,0(Rx)	;load X[i], X[i+1], X[i+2], X[i+3]
	MUL.4D F4,F4,F0	;a*X[i],a*X[i+1],a*X[i+2],a*X[i+3]
	L.4D F8,0(Ry)	;load Y[i], Y[i+1], Y[i+2], Y[i+3]
	ADD.4D F8,F8,F4	;a*X[i]+Y[i], ..., a*X[i+3]+Y[i+3]
	S.4D F8,0(Ry)	;store into Y[i], Y[i+1], Y[i+2], Y[i+3]
	DADDIU Rx,Rx,#32	;increment index to X
	DADDIU Ry,Ry,#32	;increment index to Y
	DSUBU R20,R4,Rx	;compute bound
	BNEZ R20,Loop	;check if done



UNIDADES PARA PROCESAMIENTO GRÁFICO (GPUs)

- ⊙ Las GPUs son económicas, accesibles y contienen una gran cantidad de elementos de cómputo.
- ⊙ Se han concebido con el objeto de realizar los procesamientos característicos de las aplicaciones gráficas
- ⊙ ¿Cómo poder utilizar la gran potencia de los procesadores gráficos en un espectro de aplicaciones más amplio?
- ⊙ Idea básica
 - ⊙ Modelo de ejecución heterogéneo (CPU+GPU)
 - ⊙ Desarrollar un lenguaje de programación tipo C que permita programar la GPU
 - ⊙ Unificar todo el paralelismo de la GPU bajo la abstracción denominada “CUDA Thread”
 - ⊙ Modelo de Programación: “Single Instruction (SIMD) Multiple Thread”



ARQUITECTURA DE NVIDIA GPU

- ◎ GPU NVIDIA
 - ◎ Multiprocesador compuesto por un conjunto de procesadores SIMD MT
- ◎ NVIDIA vs procesadores vectoriales
 - ◎ Similaridades
 - Funciona bien en problemas con paralelismo de datos
 - Transferencias con memoria tipo dispersar/reunir (scatter/gather)
 - Registros de máscara
 - Existencia de grandes ficheros de registros
 - ◎ Diferencias
 - No hay un procesador escalar
 - Utilización de multithreading para ocultar la latencia de memoria
 - Existencia de gran cantidad de UFs.
 - ◎ Contrasta con la reducida cantidad de UFs muy segmentadas, que es típica de los procesadores vectoriales



CUDA(COMPUTE UNIFIED DEVICE ARCHITECTURE)

- ⊙ CUDA produce código C/C++ para host y dialecto de C y C++ para la GPU
 - ⊙ Idea básica: crear un **thread** (hilo) separado para cada elemento de los vectores a procesar
 - Objetivo: generar un gran nº de hilos de cómputo independientes
 - ⊙ Los threads se agrupan en **bloques de threads**
 - El número de threads por bloque puede definirlo el programador
 - Cada bloque es ejecutado por un procesador SIMD MT de la GPU
 - Varios bloques pueden ejecutarse en paralelo sobre varios procesadores
 - ⊙ El conjunto de bloques que implementan un cálculo vectorial sobre la GPU se denomina **Grid** (malla). La ejecución del cálculo se produce con una llamada similar a una función en C:
 - **nombre_función <<<dimGrid,dimBlock>>> (... lista de parámetros ...)**
 - ⊙ **dimGrid**: nº de bloques en el Grid
 - ⊙ **dimBlock**: nº de threads por bloque
 - ⊙ Los bloques y las mallas pueden tener hasta 3 dimensiones, que se identifican con .x , .y, .z.



CUDA(COMPUTE UNIFIED DEVICE ARCHITECTURE)

◎ CUDA vs C. Ejemplo DAXPY

◎ Versión C

// Invocar DAXPY

```
daxpy(n, 2.0, x, y);
```

// DAXPY en C (bucle escalar: una iteración por elemento)

```
void daxpy(int n, double a, double *x, double *y)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
        y[i] = a*x[i] + y[i];
```

```
}
```

No hay dependencias
entre iteraciones



CUDA(Compute Unified Device Architecture)

◎ CUDA vs C. Ejemplo DAXPY

◎ Versión CUDA

// Invocar DAXPY con 256 threads por Bloque (dimBlock)

```
__host__                                     /* código para la CPU */  
int nblocks = (n+ 255) / 256;                /* cálculo del nº total de bloques en el Grid (dimGrid) */  
daxpy <<<nblocks, 256>>> (n, 2.0, x, y);
```

// DAXPY en CUDA (representa el cálculo ejecutado para un elemento)

```
__device__                                   /* código para los procesadores de la GPU */  
void daxpy(int n, double a, double *x, double *y)  
{  
    int i = blockIdx.x*blockDim.x + threadIdx.x;  
    // Si el nº elemento obtenido es mayor que el tamaño del vector, ignorar operación  
    if (i < n) y[i] = a*x[i] + y[i];  
}
```

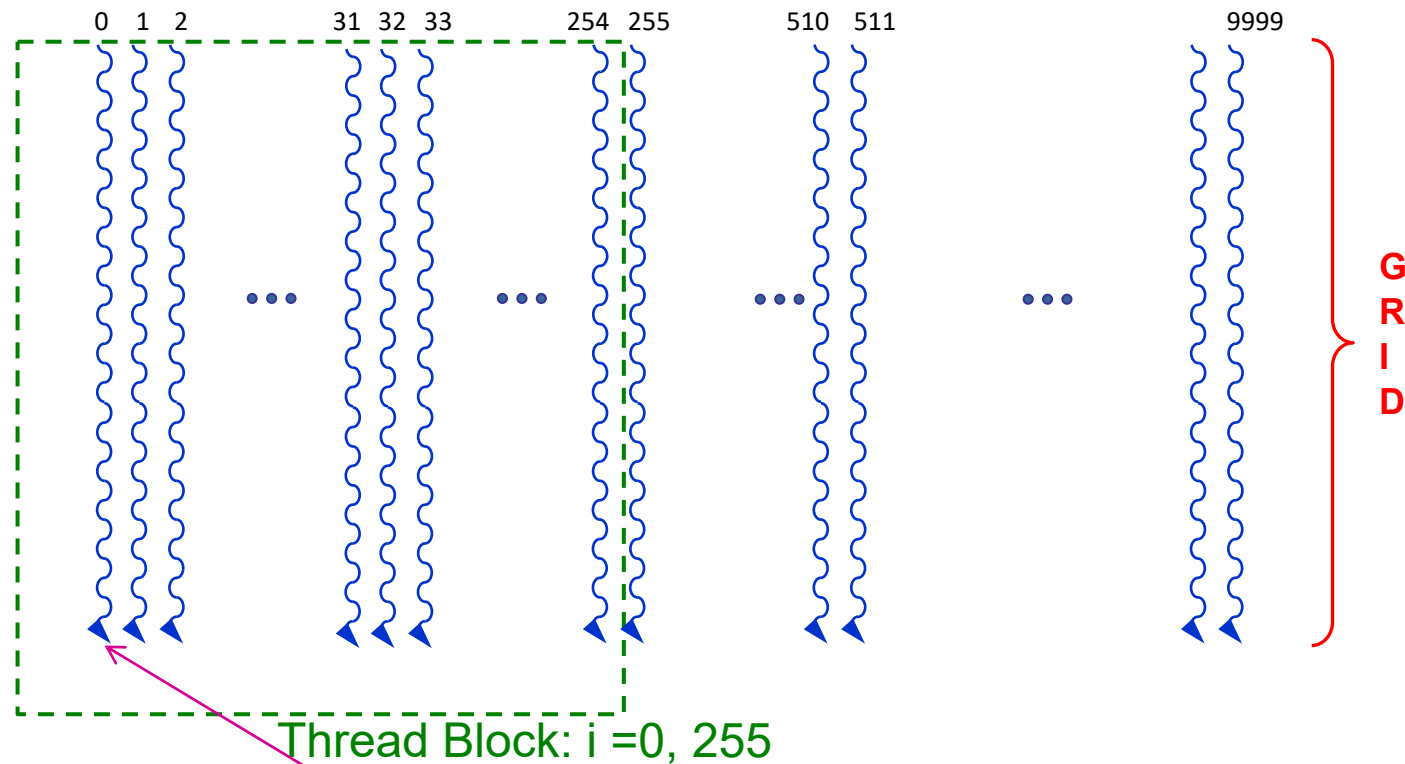
¿Qué thread soy? Calcular $i = \text{nº elemento del vector a procesar} (= \text{nº de thread})$, siendo
 $\text{nº elemento} = (\text{nº de bloque} \times \text{tamaño de bloque}) + (\text{nº de thread dentro del bloque})$



CUDA(Compute Unified Device Architecture)



Ejemplo: vectores de 10,000 componentes

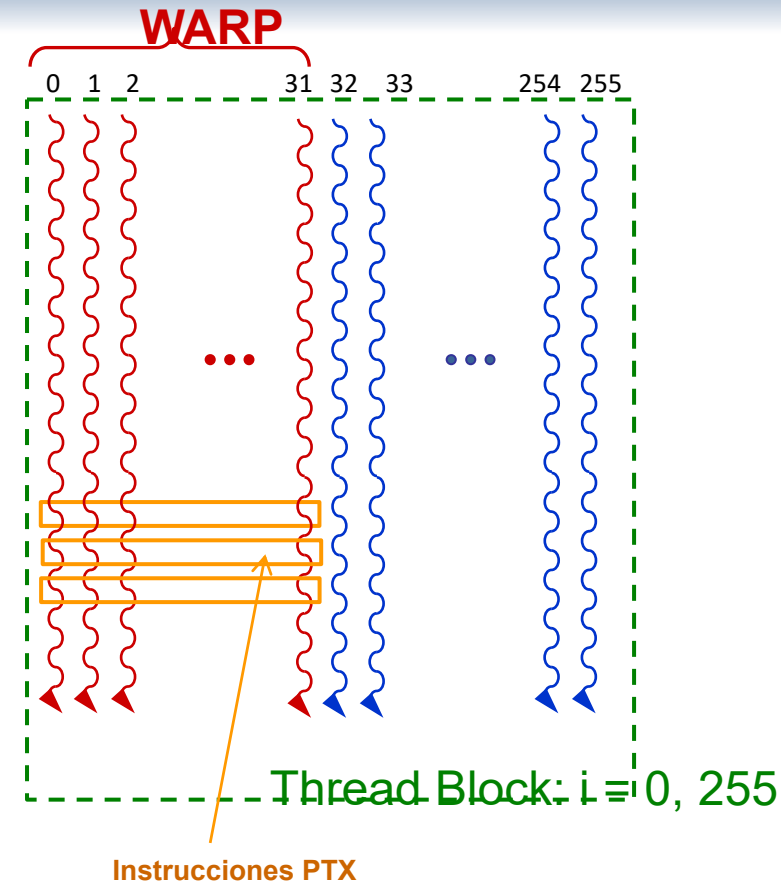


CUDA thread. Ejemplo: Calcula $Y(0) = a * X(0) + Y(0)$



THREAD BLOCKS E INSTRUCCIONES PTX

- ⦿ Cada Thread Block se ejecuta en un procesador SIMD MT de la GPU.
- ⦿ Varios procesadores SIMD MT de la GPU pueden procesar diferentes Thread Blocks en paralelo.
- ⦿ Instrucción PTX: Ejecuta un mismo cálculo sobre varios (e.g. 32) datos (Instr SIMD).
 - ⦿ Resultados afectados por registro de máscara.
- ⦿ WARP: Secuencia (thread) de instr PTX. El procesador ejecuta los WARP de un Thread Block en modo MT.
 - ⦿ En el ejemplo hay $256/32 = 8$ WARPs .
 - ⦿ Cambios de thread ocultan latencias de acceso a memoria





CÓDIGO GENERADO POR COMPILADORES DE NVIDIA

- ⊙ Instrucciones PTX (Parallel Thread Execution)
 - ⊙ Abstracción del repertorio de instrucciones hw
 - ⊙ Formato: **opcode.type dest, src1, src2, src3**
 - ⊙ Instrucción que ejecuta una operación elemental sobre múltiples datos (SIMD) utilizando todas las vías del procesador
 - ⊙ Ejemplo: un conjunto de instrucciones PTX representativas

Instruction	Example	Meaning	Comments
arithmetic .type = .s32, .u32, .f32, .s64, .u64, .f64			
add.type	add.f32 d, a, b	$d = a + b;$	
sub.type	sub.f32 d, a, b	$d = a - b;$	
mul.type	mul.f32 d, a, b	$d = a * b;$	
mad.type	mad.f32 d, a, b, c	$d = a * b + c;$	multiply-add
→ div.type	div.f32 d, a, b	$d = a / b;$	multiple microinstructions
setp.cmp.type	setp.lt.f32 p, a, b	$p = (a < b);$	compare and set predicate
numeric .cmp = eq, ne, lt, le, gt, ge; unordered cmp = equ, neu, ltu, leu, gtu, geu, num, nan			
mov.type	mov.b32 d, a	$d = a;$	move
selp.type	selp.f32 d, a, b, p	$d = p ? a : b;$	select with predicate
memory.space = .global, .shared, .local, .const; .type = .b8, .u8, .s8, .b16, .b32, .b64			
ld.space.type	ld.global.b32 d, [a+off]	$d = *(a+off);$	load from memory space
st.space.type	st.shared.b32 [d+off], a	$*(d+off) = a;$	store to memory space



CÓDIGO GENERADO POR COMPILADORES DE NVIDIA

© Ejemplo: secuencia de instrucciones PTX para una iteración del bucle DAXPY

- ⦿ Usa reg virtuales: Ri (32 bits), RDi (64 bits)
- ⦿ Asigna reg físicos en el momento de la carga del programa

shl.u32	R8, blockIdx, 8	; Thread Block ID * Block size (256 or 2 ⁸)
add.u32	R8, R8, threadIdx	; R8 = i = my CUDA Thread ID
shl.u32	R8, R8, 3	; byte offset
ld.global.f64	RD0, [X+R8]	; RD0 = X[i]
ld.global.f64	RD2, [Y+R8]	; RD2 = Y[i]
mul.f64	RD0, RD0, RD4	; Product in RD0 = RD0 * RD4 (scalar a)
add.f64	RD0, RD0, RD2	; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64	[Y+R8], RD0	; Y[i] = sum (X[i]*a + Y[i])

Ojo! Recordar que cada instrucción PTX procesa 32 elementos

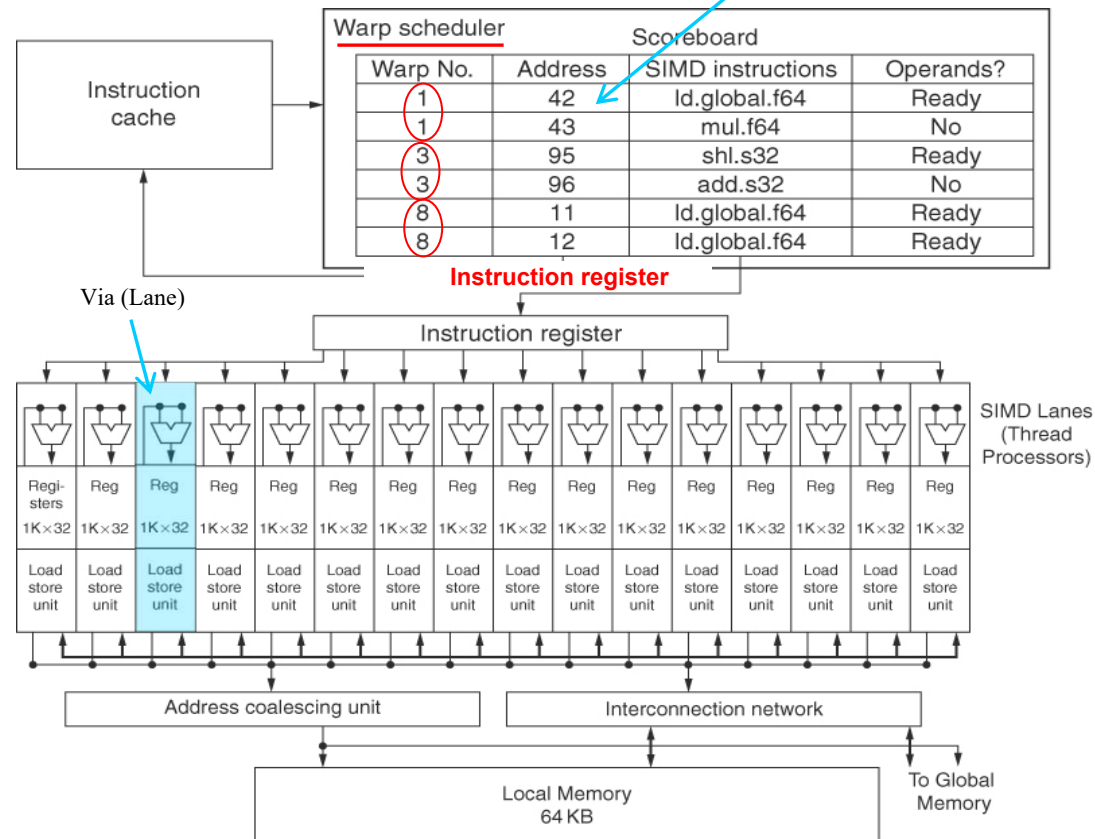


PROCESADORES DE UNA GPU

Procesador SIMD MT

Cada Warp (thread de instrucciones SIMD) tiene su PC

- Todas las vías ejecutan la misma instrucción
- Según la máscara, unas guardan el resultado y otras no

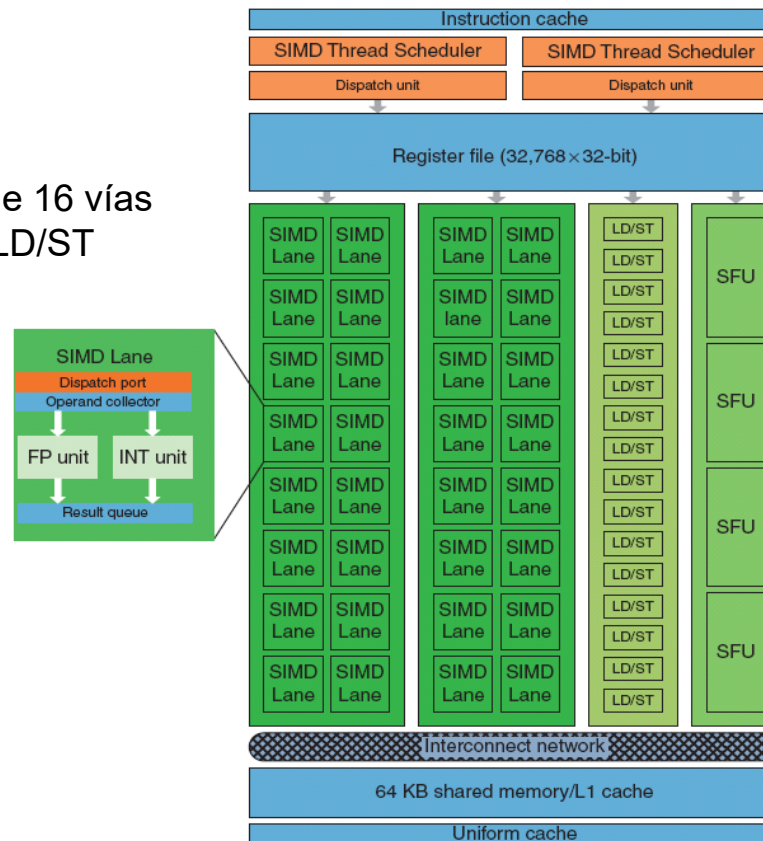




PROCESADORES DE UNA GPU

© Fermi: Esquema de un procesador SIMD MT

Dos conjuntos de 16 vías
16 unidades de LD/ST

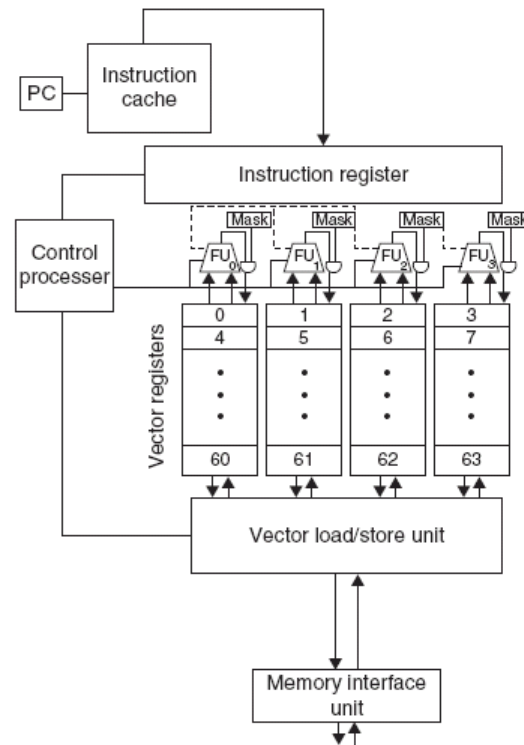


Special FU:
Calcula $\sqrt{}$, sin,
 $1/x \dots$

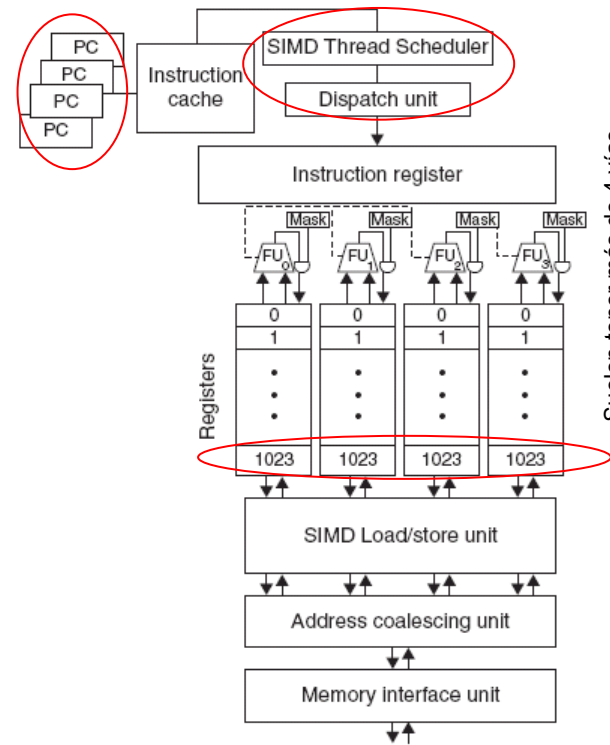


COMPARACIÓN PROCESADOR VECTORIAL - GPU

Procesador
vectorial con
cuatro vías



Procesador SIMD
MT (4 PCs) con
cuatro vías



Suelen tener más de 4 vías



REPRESENTACIÓN EN COMA FLOTANTE

- ⊙ La representación en coma flotante está basada en la **notación científica**:
 - La coma decimal no se halla en una posición fija dentro de la secuencia de bits, sino que su posición se indica como una potencia de la base:

$$\begin{array}{c} \text{signo} \\ \underbrace{+} \\ \underbrace{6.02}_{\text{mantisa}} \cdot \underbrace{10^{-23}}_{\text{base}} \end{array}$$

$$\begin{array}{c} \text{signo} \\ \underbrace{+} \\ \underbrace{1.01110}_{\text{mantisa}} \cdot \underbrace{2^{-1101}}_{\text{base}} \end{array}$$

- ⊙ En todo número en coma flotante se distinguen tres componentes:
 - **Signo**: indica el signo del número (0= positivo, 1=negativo)
 - **Mantisa**: contiene la magnitud del número (en binario puro)
 - **Exponente**: contiene el valor de la potencia de la base (sesgado)
 - La **base** queda implícita y es común a todos los números, la más usada es 2



REPRESENTACIÓN EN COMA FLOTANTE

- ⊙ El **valor** de la secuencia de bits $(s, e_{p-1}, \dots, e_0, m_{q-1}, \dots, m_0)$ es: $(-1)^s \cdot V(m) \cdot 2^{V(e)}$
- ⊙ Dado que un mismo número puede tener varias representaciones

$$(0.110 \cdot 2^5 = 110 \cdot 2^2 = 0.0110 \cdot 2^6)$$

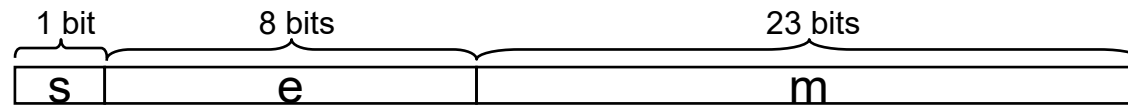
los números suelen estar normalizados:

- ⊙ Un número está normalizado si tiene la forma $1.xx... \cdot 2^{xx...}$ (ó $0.1xx... \cdot 2^{xx...}$)
- ⊙ Dado que los números normalizados en base 2 tienen siempre un 1 a la izquierda, éste suele quedar implícito (pero debe ser tenido en cuenta al calcular el valor de la secuencia)



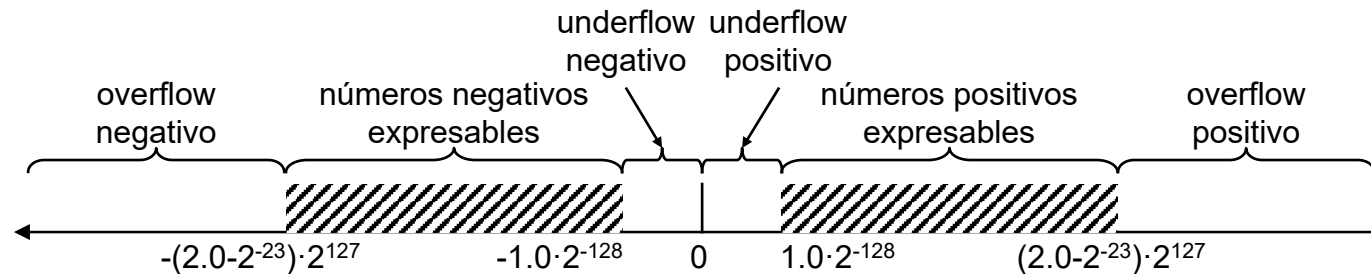
REPRESENTACIÓN EN COMA FLOTANTE

- Sea el siguiente formato de coma flotante de 32 bits (base 2, normalizado)

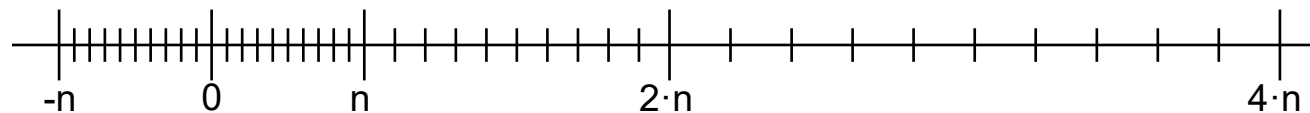


- El rango de valores representable por cada uno de los campos es:

- Exponente** (8 bits con sesgo de 128) : $-128 \dots +127$
- Mantisa** (23 bits normalizados) : los valores binarios representables oscilan entre $1.00\dots00$ y $1.11\dots11$, es decir entre 1 y $2-2^{-23}$ (2-ulp) ($1.11\dots1 = 10.00\dots0 - 0.0\dots1$)



- Obsérvese que la cantidad de números representables es 2^{32} (igual que en coma fija). Lo que permite la representación en coma flotante es ampliar el rango representable a costa de aumentar el espacio entre números representable (un espacio que no es uniforme).





- ⊙ **2 formatos** con signo explícito, representación sesgada del exponente (sesgo igual a $(2^{n-1}-1)$), mantisa normalizada con un 1 implícito (1.M) y base 2.
 - ***precisión simple*** (32 bits): 1 bit de signo, 8 de exponente, 23 de mantisa
 - $1.0 \cdot 2^{-126} \dots (2-2^{-23}) \cdot 2^{127} = 1.2 \cdot 10^{-38} \dots 3.4 \cdot 10^{38}$
 - ***precisión doble*** (64 bits): 1 bit de signo, 11 de exponente, 52 de mantisa
 - $1.0 \cdot 2^{-1022} \dots (2-2^{-52}) \cdot 2^{1023} = 2.2 \cdot 10^{-308} \dots 1.8 \cdot 10^{308}$
- ⊙ **2 formatos** ampliados para cálculos intermedios (43 y 79 bits).



⦿ Codificaciones con significado especial

- **Infinito** ($e=255, m=0$): representan cualquier valor de la región de overflow
- **NaN** (*Not-a-Number*) ($e=255, m>0$): se obtienen como resultado de operaciones inválidas
- **Número denormalizado** ($e=0, m>0$): es un número sin normalizar cuyo bit implícito se supone que es 0. Al ser el exponente 0, permiten representar números en las regiones de underflow. El valor del exponente es el del exponente más pequeño de los números no denormalizados: -126 en precisión simple y -1022 en doble.
- **Cero** ($e=0, m=0$): número no normalizado que representa al cero (en lugar de al 1)

⦿ Excepciones:

- **Operación inválida**: $\infty \pm \infty$, $0 \times \infty$, $0 \div 0$, $\infty \div \infty$, $x \bmod 0$, $\sqrt{x}$ cuando $x < 0$, $x = \infty$
- **Inexacto**: el resultado redondeado no coincide con el real
- **Overflow y underflow**
- **División por cero**



⊙ Ejemplo:

0	0110 1000	101 0101 0100 0011 0100 0010
---	-----------	------------------------------

⊙ Signo: 0 => positivo

⊙ Exponente:

⊙ $0110\ 1000_{\text{dos}} = 104_{\text{diez}}$

⊙ Eliminación del sesgo: $104 - 127 = -23$

⊙ Mantisa:

$$1 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 0 \times 2^{-4} + 1 \times 2^{-5} + \dots$$
$$= 1 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-9} + 2^{-11} + 2^{-13} + 2^{-15} + 2^{-17} + 2^{-19} + 2^{-21} = 1.0 + 0.666115$$

Representa: $1.666115_{\text{diez}} \times 2^{-23} \sim 1.986 \times 10^{-7}$



⦿ Redondeo:

- ⦿ El estándar exige que el resultado de las operaciones sea el mismo que se obtendría si se realizasen con precisión absoluta y después se redondease.
- ⦿ Hacer la operación con precisión absoluta no tiene sentido pues se podrían necesitar operandos de mucha anchura.
- ⦿ **Existen 4 modos de redondeo:**
 - Redondeo al **más cercano** (al par en caso de empate)
 - Redondeo a **más infinito** (por exceso)
 - Redondeo a **menos infinito** (por defecto)
 - Redondeo a **cero** (truncamiento)



- ⊙ Al realizar una operación ¿cuántos bits adicionales se necesitan para tener la precisión requerida?
- ⊙ Un bit **r** para el redondeo
- ⊙ Un bit **s** (sticky) para determinar, cuando $r=1$, si el número está por encima de 0,5

Operaciones de redondeo

Tipo de redondeo	Signo del resultado ≥ 0	Signo del resultado < 0
$-\infty$		+1 si (r or s)
$+\infty$	+1 si (r or s)	
0		
Más próximo	+1 si (r and p_0) or (r and s)	+1 si (r and p_0) or (r and s)



IEEE 754: SUMA

- Objetivo: Sumar en formato punto fijo de números con el exponente alineado

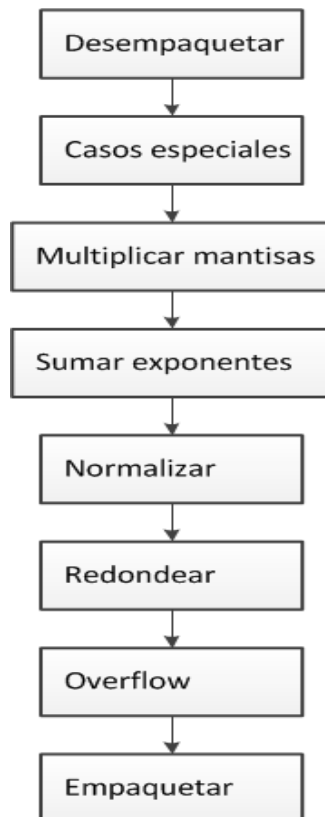


Ejemplo:

$$\begin{array}{rcl} + \begin{array}{r} 9.555 \cdot 10^4 \\ 9.996 \cdot 10^7 \end{array} & \longrightarrow & + \begin{array}{r} 0.009555 \cdot 10^7 \\ 9.996000 \cdot 10^7 \end{array} \\ & \text{alinear} & \\ \hline & & 10.005555 \cdot 10^7 \longrightarrow 1.0005555 \cdot 10^8 \longrightarrow 1.001 \cdot 10^8 \\ & & \text{normalizar} \qquad \qquad \text{redondeo} \end{array}$$



IEEE 754: MULTIPLICACIÓN



Ejemplo:

$$\begin{array}{r} 4.555 \cdot 10^4 \\ \times 2.996 \cdot 10^7 \\ \hline 13.646780 \cdot 10^{11} \end{array} \xrightarrow{\text{normalizar}} 1.3646780 \cdot 10^{12} \xrightarrow{\text{redondeo}} 1.365 \cdot 10^{12}$$